Two-scale stochastic simulation of epidemics on community networks with aggregate inter-community coupling

Yulian Kuryliak[a,*], Michael T. M. Emmerich[b]

[a]Institute of Computer Science and Information Technologies, Lviv Polytechnic National University, 12 Stepan Bandera Street, Lviv, 79000, Ukraine

[b]Faculty of Information Technology, University of Jyväskylä, P.O. Box 35 (Agora), Jyväskylä, FI-40014, Finland

1. Introduction

Epidemics spread at more than one scale. Inside a community, infection may pass through a detailed network of contacts between people (Keeling and Eames, 2005; Pastor-Satorras et al., 2015; Kiss et al., 2017). Between communities, the available data are often much less detailed. For example, they may give only passenger numbers, commuting flows, or a weighted link between two places (Balcan et al., 2009; Barbosa et al., 2018). We therefore need a model that can use a detailed network inside each community and simpler data between communities.

This situation is common in practice. A local contact network may be observed, reconstructed, or generated to match known properties such as clustering and mean shortest-path length (Eubank et al., 2004; Danon et al., 2011). In contrast, links between communities may be known only through transport, migration, commuting, or mobile-phone data (Barbosa et al., 2018; Chang et al., 2021). Such data can tell us the total strength of connection between two communities, but not which people form the connection. We may therefore know the total coupling without knowing the bridge nodes or their infection states.

If every contact is known, a continuous-time Markov chain (CTMC) can describe the epidemic, and a Gillespie-type method can simulate it exactly (Gillespie, 1977; Gibson and Bruck, 2000). Efficient methods are also available for temporal, large, and sparse networks (Vestergaard and Génois, 2015; Kuryliak et al., 2025). However, these methods still need the individual contacts that may transmit infection. Metapopulation models use aggregate flows between communities, but usually also replace the detailed local network by population counts (Colizza et al., 2007; Colizza and Vespignani, 2008; Balcan et al., 2010; Ajelli et al., 2010). Our problem lies between these two cases: we want to keep node-level stochastic spread inside every community while using only aggregate information between communities.

The idea of a two-scale simulator was presented in an earlier conference paper (Kuryliak and Emmerich, 2025). It runs a separate local simulator for each community. The macro layer uses the current community states and connection weights to generate infections arriving from another community. In this article, we describe the method in detail and study its approximation error. In particular, we examine what is lost when real cross-community edges and their endpoints are replaced by the total connection weight, and an imported infection is placed on a randomly selected susceptible node.

Our main question is: when is this aggregate connection sufficient to reproduce the quantity of interest in the full-network CTMC? We study infection arrival times, their distributions, and whether transmission is possible through the real contacts. Similar arrival times do not guarantee a possible transmission route: the two-scale model can infect another community while all real bridge nodes in the source community are still susceptible. To test this, we use synthetic networks in which every contact is known. Full-network Gillespie simulation receives the whole network, whereas the two-scale method receives only the local networks and the total weights of the cross-community connections.

This article makes four contributions.

*Corresponding author

yulian.a.kuryliak@lpnu.ua (Y. Kuryliak); michael.t.m.emmerich@jyu.fi (M.T.M. Emmerich)

ORCID(s): 0000-0002-6600-2637 (Y. Kuryliak); 0000-0002-7342-2090 (M.T.M. Emmerich)

1. We describe how the simulator works: each community is simulated separately, and a shared scheduler determines infections between them. We explain how it coordinates the local simulators on time intervals of fixed length (Section 3, Algorithm 1).
2. We show which network the method simulates exactly. It is exact for a *surrogate network*: the real cross-community edges are replaced by weak connections between all node pairs, preserving the total weight (Proposition 1). We then identify the sources of error and analyse a single export of infection (Proposition 2).
3. We compare the two-scale method with full-network Gillespie simulation for transmission to one neighbouring community and for spread along chains of communities. We measure differences using the relative shift in the median time of first infection and the Wasserstein distance between the distributions of these times (Sections 5 and 6).
4. We propose an empirical indicator of accuracy for our benchmarks. It compares the spread of infection times inside a community with the typical transmission time through one cross-community edge (Section 6.6).

We consider susceptible–infected (SI) dynamics on static synthetic networks and use one fixed synchronisation step. Section 2 explains the relation to earlier methods. Section 3 describes the simulator. Section 4 explains what the aggregate interface preserves and loses. Sections 5 and 6 present the experiments and results. Section 7 discusses use, cost, and limitations.

2. Related work

Simulation of stochastic events. Gillespie's direct method draws the next event and its time from the current event rates (Gillespie, 1977). The next reaction method keeps a separate clock for each possible event (Gibson and Bruck, 2000). Both methods are exact for the CTMC they simulate. When an event rate changes with time, its rate can be accumulated until it reaches a random threshold (Ethier and Kurtz, 1986). Anderson (2007) used this idea for systems with time-dependent rates and delays. Our macro clock uses the same idea, but its rate changes because the local epidemic states change. Network simulators can save work by updating only the rates affected by the latest event (Vestergaard and Génois, 2015; Kuryliak et al., 2025). Our local simulators use the implementation of Kuryliak et al. (2025).

Some multiscale methods speed up chemical-reaction models by approximating reactions that occur much faster than others (Haseltine and Rawlings, 2002; Cao et al., 2005). Our method removes a different kind of detail. It keeps the local infection events, but does not know which individual cross-community edges are currently able to transmit. Even so, the relative speeds of local spread and cross-community transmission help explain its error.

Models with several communities. Metapopulation models represent each subpopulation by counts and connect the subpopulations by mobility flows. They have been used to study global spread and the conditions for invasion (Colizza et al., 2007; Colizza and Vespignani, 2008; Balcan et al., 2009, 2010). Detailed mobility data can also be combined with local transmission models (Chang et al., 2021). Agent-based models instead represent individuals throughout the model (Eubank et al., 2004; Kerr et al., 2021). Ajelli et al. (2010) compared these two broad approaches. The method studied here keeps individual nodes and contacts inside each community, but replaces the unknown individual contacts between communities by one aggregate stochastic interface.

Two levels of mixing and co-simulation. Some epidemic models combine contacts in small local groups with uniform mixing at a wider level (Ball et al., 1997). Our surrogate network is related to this idea, but uniform mixing is used only between connected pairs of communities, and its strength is set by their aggregate weight.

The implementation also has features of co-simulation. In co-simulation, separate subsystem solvers move forward on a shared time grid and exchange a limited amount of information (Gomes et al., 2018). Here, the local community simulators are those subsystems. They send their infected and susceptible counts to the macro layer. In this implementation, those exchanged counts are used to draw a cross-community infection time and then to choose its receiving community and target node. The scheduler can therefore treat the local simulators as black boxes.

3. The two-scale simulation method

The experiments use one implementation of the two-scale simulator, in which the local simulators exchange information with the macro scheduler at fixed time intervals. To understand its error, we also consider a process with the same rates but exact timing of every event. We call this theoretical description the *ideal process*. It separates the error caused by changing the network from the additional error caused by the time grid. The ideal process is exact for the surrogate network defined later; this does not make it exact for the original physical network. Below, we explain the reference model, the available data, and the steps of the implemented simulator.

3.1. Full-network reference model

Let $G = (V, E)$ be the contact network. Its nodes are divided into M non-overlapping communities:

$$V = V_1 \cup V_2 \cup \cdots \cup V_M, \qquad N_i = |V_i|. \tag{1}$$

Here, V_i is the node set of community i, and N_i is its size. We write E_{ii} for contacts inside community i and E_{ij} for contacts from nodes in community i to nodes in community j. We treat each contact as an oriented transmission channel. An undirected contact therefore gives two channels, one in each direction.

Every node is susceptible (S), infected (I), or removed (R). The full network state at time t is $X(t)$, and the state of community i is $X_i(t)$. Let $S_i(t)$ and $I_i(t)$ be the sets of susceptible and infected nodes in that community. Their sizes are $s_i(t)$ and $y_i(t)$, and their fractions are $\bar{s}_i(t) = s_i(t)/N_i$ and $\bar{y}_i(t) = y_i(t)/N_i$. Below, x denotes a particular network state; $x^{v:I}$ is the same state after node v becomes infected, and $x^{u:R}$ is the state after node u is removed.

The full model follows two simple rules. A susceptible node v receives infection from each infected neighbour u at rate β_{uv}. An infected node is removed at rate γ. We can write both rules as

$$q_v(x) = \sum_{\substack{u:(u,v)\in E \\ x_u = I}} \beta_{uv} \quad (x_v = S), \qquad r_u(x) = \gamma \quad (x_u = I). \tag{2}$$

Here, q_v is the total infection rate of node v, and r_u is the removal rate of node u. For a non-susceptible node, $q_v = 0$; for a non-infected node, $r_u = 0$. A rate describes how often an event occurs: for a constant rate q, the mean waiting time is $1/q$. For a cross-community channel, we write $\beta_{uv} = \beta_{\text{out}} w_{uv}$, where w_{uv} is its contact weight. We call exact simulation of these full-network rates Full. All experiments in this article use SI dynamics, so $\gamma = 0$.

The waiting time between state transitions is exponential and is determined by the total rate of all events in the current state x, which is

$$a(x) = \sum_v q_v(x) + \sum_u r_u(x).$$

If $a(x) > 0$, the waiting time to the next event is exponentially distributed with rate $a(x)$ and mean $1/a(x)$. Infection of node v is selected with probability $q_v(x)/a(x)$, and removal of node u with probability $r_u(x)/a(x)$. After the event, the rates are updated. If $a(x) = 0$, no further event can occur.

3.2. Aggregate interface

The two-scale simulator, denoted TwoScale, knows every local graph $G_i = (V_i, E_{ii})$ and its rates. It does not know the individual cross-community channels in E_{ij}. For each ordered pair of communities, it knows only the total weight

$$W_{ij} = \sum_{(u,v)\in E_{ij}} w_{uv}, \qquad i \neq j. \tag{3}$$

If all weights equal one, W_{ij} is the number of directed channels from V_i to V_j. In an application, this total may instead come from transport, migration, mobility, or contact data. The macro layer also receives the current susceptible and infected counts from each local simulator. It does not know which nodes are endpoints of real cross-community edges. Table 1 gives the main difference, and Figure 1 shows it visually.

Information or action	FULL	TWOSCALE
Local contacts, rates, and node states	known	known by local simulators
Individual cross-community edges and bridge nodes	known	unknown
Total cross-community weight W_{ij}	obtained from the edges	given directly
Cross-community event rate	uses states of real edge endpoints	uses $\bar{y}_i$, $\bar{s}_j$, and W_{ij}
Target of a cross-community infection	endpoint of the active edge	a susceptible node drawn by the target rule
Number of possible SI infections through one bundle	at most the number of its channels	at most s_j target infections

Table 1: Information used by the two models. The "target rule" in TWOSCALE is uniform among susceptible nodes in all experiments in this article.

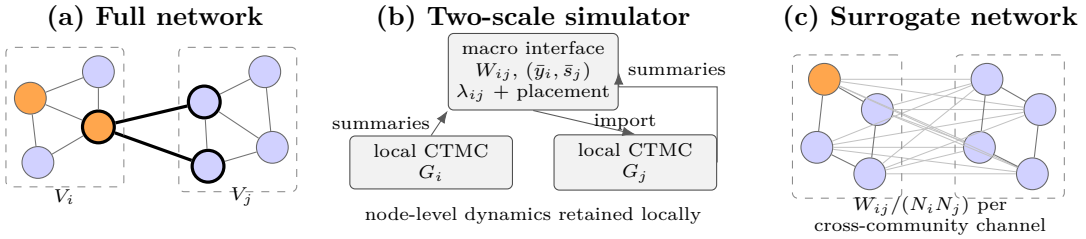


Figure 1: Three views of the interface. (a) In the full network, infection can cross only through the shown physical edges and their bridge nodes. (b) The two-scale simulator keeps a separate local simulator for each community. The macro layer receives community counts, uses W_{ij} to generate a cross-community infection, and places it on a uniformly selected susceptible node. (c) The ideal two-scale process is equivalent to a surrogate network in which every ordered node pair across the two communities has a weak connection. The connections together have total weight W_{ij}. Orange nodes are infected and blue nodes are susceptible.

3.3. Local simulators

Each community has a separate Gillespie simulator for its local graph G_i. It applies the infection and removal rates in (2) using only the local edges E_{ii}. The implementation updates only rates affected by the latest event, following Kuryliak et al. (2025). Each local simulator is independent and has its own random-number stream.

The macro layer can advance a local simulator to a specified time. The simulator applies the local events up to that time. The macro layer can then import an infection by changing one selected susceptible node to I. This changes the local rates. The implementation keeps the unused part of the random threshold for the next local event and recalculates its waiting time using the new total rate.

To recalculate the time of the next local event after an import, the simulator retains the remaining random threshold. Let $E_{\mathrm{loc}} \sim \mathrm{Exp}(1)$ be the threshold drawn after the previous local event, and let A_{loc} be the integral of the total local rate from that event to the import time. After the import, the total local rate is a_{new}. If $a_{\mathrm{new}} > 0$, the remaining waiting time is

$$\Delta t = \frac{E_{\mathrm{loc}} - A_{\mathrm{loc}}}{a_{\mathrm{new}}}.$$

Thus, an import changes the waiting time, but not the random threshold itself. If another import occurs first, the accumulated rate is updated and the waiting time is recalculated again. If $a_{\mathrm{new}} = 0$, no local event can occur until the rate becomes positive.

The event type and its node are selected using the updated rates. A new local threshold is drawn only if there is no existing one. Each local clock has its own threshold, separate from the macro threshold described in Section 3.5. For prescribed import times and target nodes, the local simulator follows the CTMC rules of

its own network. The approximation comes from the way imports are generated and their target nodes are selected.

3.4. Macro layer

The macro layer represents each community by its current fractions and represents each directed connection by W_{ij}. Its rate for an infection from community i to community j is

$$\lambda_{ij}(t) = \beta_{\text{out}} W_{ij} \bar{y}_i(t) \bar{s}_j(t), \qquad \Theta(t) = \sum_{i \neq j} \lambda_{ij}(t). \tag{4}$$

The factors have a direct meaning: $\beta_{\text{out}} W_{ij}$ gives the total connection strength, $\bar{y}_i$ is the infected fraction in the source, and $\bar{s}_j$ is the susceptible fraction in the receiving community. The sum Θ is the rate of any cross-community infection. Calculating the waiting time to the next event when Θ varies with time requires special treatment (Section 3.5).

When such an event occurs, first select the source and receiving communities (i, j) with probability λ_{ij}/Θ. Then choose one susceptible node uniformly in the receiving community j and infect it. If there are 20 susceptible nodes, each has probability $1/20$. This selection does not use real bridge nodes, because the aggregate interface does not know them. In later formulas, we write this target probability as $\Pi_{ij}(v \mid x) = 1/s_j$ for $v \in S_j$, where x is the current network state. Thus, Π simply denotes the probability of choosing a particular susceptible node. Other target rules can be used if bridge-node information is available.

This aggregate rate is generally an approximation to transmission through the real physical edges. For fixed community counts, it is the expected rate under uniformly random selection of the edge endpoints, before their particular identities, states, and epidemic history are taken into account. This does not guarantee the correct rate for an epidemic already realised on specific edges. For uniform all-to-all coupling, it is the actual rate, as explained in Section 4.

3.5. Timing and synchronisation

After a macro event at time t_0, draw a random threshold $E \sim \text{Exp}(1)$: an exponential random number with mean one. Starting from zero, accumulate the total macro rate Θ:

$$A(t) = \int_{t_0}^{t} \Theta(s)\, ds, \qquad A(t_0) = 0. \tag{5}$$

The next macro event occurs at the first time t^* when $A(t^*) = E$.

In words, the area under the rate curve grows until it first reaches the random threshold. If it never reaches the threshold, no further macro event occurs. This is the standard cumulative-rate construction (Ethier and Kurtz, 1986; Anderson, 2007). If Θ is constant, the waiting time is E/Θ. Drawing $E = -\ln U$, with U uniform on $(0, 1)$, gives the familiar Gillespie waiting time. When Θ changes after a local event, we keep the same threshold and continue accumulating with the new rate. We draw a new macro threshold only after a macro event. Exact calculation of its first crossing defines the ideal process; the time-grid calculation below approximates that crossing. Section 4.1 shows that the ideal process is exact for the surrogate network.

The scheduler divides time into intervals of length τ. On each interval, it approximates the varying macro rate by its value at the midpoint. Let A be the amount already accumulated since the previous macro event. For the current interval $[t, t_g)$, copy every local simulator, including its random-number state, and advance each copy to the midpoint. The copies give an estimate Θ_{mid} of the total macro rate. The estimated increase over the remaining interval is $\Theta_{\text{mid}}(t_g - t)$. If A plus this increase is below E (the random macro threshold), the real simulators move to t_g. Otherwise, estimate the event time by

$$t^* = t + \frac{E - A}{\Theta_{\text{mid}}}. \tag{6}$$

Here, E is the random macro threshold and A is the amount already accumulated toward it. The numerator $E - A$ is what remains; dividing by the estimated rate gives the remaining waiting time. For example, if $E = 1$, $A = 0.4$, and $\Theta_{\text{mid}} = 0.2$, the estimated wait is three time units. The scheduler advances the real

simulators to t^*, selects the source and receiving communities using their states at that time, chooses the susceptible target, and imports the infection. Algorithm 1 gives the full sequence.

Algorithm 1 Grid-synchronised two-scale simulation of one realisation.

Input: local graphs G_i, weights W_{ij}, rates, step τ, horizon T_{end}, and initial state.

1. Set $t = 0$ and accumulated rate $A = 0$; draw $E \sim \text{Exp}(1)$.
2. While $t < T_{\text{end}}$ and infection can still spread, set $t_g = \min(t + \tau, T_{\text{end}})$ and process the interval up to t_g.
3. Copy every local simulator, including its random-number state. Advance the copies to the midpoint of the remaining interval and calculate Θ_{mid} from (4).
4. If $A + \Theta_{\text{mid}}(t_g - t) < E$, advance the real simulators to t_g, add this amount to A, set $t = t_g$, and return to step 2.
5. Otherwise, calculate t^* from (6) and advance the real simulators to t^*.
6. Select (i, j) with probability $\lambda_{ij}(t^*)/\Theta(t^*)$. Select v uniformly from $S_j(t^*)$ and import infection at v. Update the receiving local simulator, keeping its unused exponential threshold and selecting the local event from the new rates.
7. Set $t = t^*$ and $A = 0$, draw a new $E \sim \text{Exp}(1)$, and process the rest of the current interval in the same way.

The copied random-number states are important. Unless an import occurs first, a copied simulator follows the same path that the real simulator will follow to the midpoint. The midpoint estimate is exact on an interval with no local event. Its timing error and its look-ahead of at most $\tau/2$ decrease to zero as τ decreases.

The time grid is needed because the scheduler treats local simulators as black boxes. If they also report their next local-event times, the scheduler can merge those times and accumulate the piecewise-constant value of Θ exactly. That event-driven version removes the numerical synchronisation approximation. The experiments in this article use the implemented grid version with $\tau = 1$.

3.6. Computational structure and intended use

Full stores all local and cross-community channels, and its work per event grows with node degree (Kuryliak et al., 2025). TwoScale stores the local graphs and an $M \times M$ matrix of aggregate weights. The conference paper discusses this reduction in the information stored for cross-community contacts (Kuryliak and Emmerich, 2025). Its importance depends on the problem. Replacing many individual cross-community edges by one weight can reduce storage. If only a few such edges exist, this saving may be small, and the local graphs still have to be stored.

The separate local simulators also provide an opportunity for parallel execution. Once the scheduler has chosen the next target time (t_g or t^*), the local simulators can advance independently to that same time. The midpoint copies can also be processed in parallel. The macro layer must then collect the counts and apply any scheduled import before continuing. The implementation studied here advances the local simulators sequentially; parallel execution within one run is a possible extension, not a measured speedup. Local simulators can also use different community-specific settings.

The grid scheduler adds work by copying local states and coordinating the communities. Therefore, neither lower memory use nor faster execution is guaranteed. The method is useful when individual cross-community contacts are unavailable or too numerous to represent explicitly.

References

Ajelli, M., Gonçalves, B., Balcan, D., Colizza, V., Hu, H., Ramasco, J.J., Merler, S., Vespignani, A., 2010. Comparing large-scale computational approaches to epidemic modeling: Agent-based versus structured metapopulation models. BMC Infectious Diseases 10, 190. doi:10.1186/1471-2334-10-190.

Anderson, D.F., 2007. A modified next reaction method for simulating chemical systems with time dependent propensities and delays. The Journal of Chemical Physics 127, 214107. doi:10.1063/1.2799998.

Balcan, D., Colizza, V., Gonçalves, B., Hu, H., Ramasco, J.J., Vespignani, A., 2009. Multiscale mobility networks and the spatial spreading of infectious diseases. Proceedings of the National Academy of Sciences 106, 21484–21489. doi:10.1073/pnas.0906910106.

Balcan, D., Colizza, V., Gonçalves, B., Hu, H., Ramasco, J.J., Vespignani, A., 2010. Modeling the spatial spread of infectious diseases: The global epidemic and mobility computational model. Journal of Computational Science 1, 132–145. doi:10.1016/j.jocs.2010.07.002.

Ball, F., Mollison, D., Scalia-Tomba, G., 1997. Epidemics with two levels of mixing. The Annals of Applied Probability 7, 46–89.

Barbosa, H., Barthelemy, M., Ghoshal, G., James, C.R., Lenormand, M., Louail, T., Menezes, R., Ramasco, J.J., Simini, F., Tomasini, M., 2018. Human mobility: Models and applications. Physics Reports 734, 1–74. doi:10.1016/j.physrep.2018.01.001.

Cao, Y., Gillespie, D.T., Petzold, L.R., 2005. The slow-scale stochastic simulation algorithm. The Journal of Chemical Physics 122, 014116.

Chang, S., Pierson, E., Koh, P.W., Gerardin, J., Redbird, B., Grusky, D., Leskovec, J., 2021. Mobility network models of COVID-19 explain inequities and inform reopening. Nature 589, 82–87. doi:10.1038/s41586-020-2923-3.

Colizza, V., Pastor-Satorras, R., Vespignani, A., 2007. Reaction–diffusion processes and metapopulation models in heterogeneous networks. Nature Physics 3, 276–282. doi:10.1038/nphys560.

Colizza, V., Vespignani, A., 2008. Epidemic modeling in metapopulation systems with heterogeneous coupling pattern: Theory and simulations. Journal of Theoretical Biology 251, 450–467. doi:10.1016/j.jtbi.2007.11.028.

Danon, L., Ford, A.P., House, T., Jewell, C.P., Keeling, M.J., Roberts, G.O., Ross, J.V., Vernon, M.C., 2011. Networks and the epidemiology of infectious disease. Interdisciplinary Perspectives on Infectious Diseases 2011, 284909. doi:10.1155/2011/284909.

Ethier, S.N., Kurtz, T.G., 1986. Markov Processes: Characterization and Convergence. Wiley, New York.

Eubank, S., Guclu, H., Kumar, V.S.A., Marathe, M.V., Srinivasan, A., Toroczkai, Z., Wang, N., 2004. Modelling disease outbreaks in realistic urban social networks. Nature 429, 180–184. doi:10.1038/nature02541.

Gibson, M.A., Bruck, J., 2000. Efficient exact stochastic simulation of chemical systems with many species and many channels. The Journal of Physical Chemistry A 104, 1876–1889.

Gillespie, D.T., 1977. Exact stochastic simulation of coupled chemical reactions. The Journal of Physical Chemistry 81, 2340–2361. doi:10.1021/j100540a008.

Gomes, C., Thule, C., Broman, D., Larsen, P.G., Vangheluwe, H., 2018. Co-simulation: A survey. ACM Computing Surveys 51, 49:1–49:33.

Haseltine, E.L., Rawlings, J.B., 2002. Approximate simulation of coupled fast and slow reactions for stochastic chemical kinetics. The Journal of Chemical Physics 117, 6959–6969.

Keeling, M.J., Eames, K.T.D., 2005. Networks and epidemic models. Journal of The Royal Society Interface 2, 295–307. doi:10.1098/rsif.2005.0051.

Kerr, C.C., Stuart, R.M., Mistry, D., Abeysuriya, R.G., Rosenfeld, K., Hart, G.R., Núñez, R.C., Cohen, J.A., Selvaraj, P., Hagedorn, B., George, L., Jastrzebski, M., Izzo, A.S., Fowler, G., Palmer, A., Delport, D., Scott, N., Kelly, S.L., Bennette, C.S., Wagner, B.G., Chang, S.T., Oron, A.P., Wenger, E.A., Panovska-Griffiths, J., Famulare, M., Klein, D.J., 2021. Covasim: An agent-based model of COVID-19 dynamics and interventions. PLoS Computational Biology 17, e1009149. doi:10.1371/journal.pcbi.1009149.

Kiss, I.Z., Miller, J.C., Simon, P.L., 2017. Mathematics of Epidemics on Networks: From Exact to Approximate Models. Springer, Cham. doi:10.1007/978-3-319-50806-1.

Kuryliak, Y., Emmerich, M., 2025. Towards complexity reduction of large-scale epidemic simulation in two-scale networks, in: Proceedings of the Modern Machine Learning Technologies Workshop (MoMLeT 2025), pp. 327–336. URL: https://ceur-ws.org/Vol-4004/.

Kuryliak, Y., Emmerich, M.T.M., Dosyn, D., 2025. Simulating epidemic peak dynamics on complex networks using efficient gillespie algorithms. Infection, Genetics and Evolution 132, 105768. doi:10.1016/j.meegid.2025.105768.

Pastor-Satorras, R., Castellano, C., Van Mieghem, P., Vespignani, A., 2015. Epidemic processes in complex networks. Reviews of Modern Physics 87, 925–979. doi:10.1103/RevModPhys.87.925.

Vestergaard, C.L., Génois, M., 2015. Temporal gillespie algorithm: Fast simulation of contagion processes on time-varying networks. PLoS Computational Biology 11, e1004579. doi:10.1371/journal.pcbi.1004579.